

# Análisis de Algoritmos en Python

Trabajo Integrador - Programación I

## Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía
8. Anexos

## Alumnos:

Benjamin Ataide – benjaminatai@gmail.com

Natanael Caliva – [calivanatanaelmdz@gmail.com](mailto:calivanatanaelmdz@gmail.com)

**Materia:** Programación I

**Profesor:** Cinthia Rigoni

**Fecha de Entrega:** 9 de junio de 2025

# Introducción

A veces, cuando escribimos código, nos concentramos en que simplemente funcione. Si devuelve el resultado correcto, parece suficiente. Pero a medida que los programas tienen que manejar más datos o funcionar en situaciones más exigentes, empezamos a notar que no todos los algoritmos rinden igual. Algunos tardan demasiado o consumen mucha memoria, y eso puede convertirse en un problema real.

En este trabajo quisimos explorar justamente eso: cómo elegir algoritmos que no solo resuelvan un problema, sino que lo hagan de forma eficiente. Para eso, tomamos un ejemplo simple pero muy común en la vida real: encontrar el producto más vendido dentro de una lista de ventas.

Implementamos dos soluciones distintas en Python: una más básica y otra optimizada. Después, analizamos su comportamiento tanto desde la teoría (usando la notación Big-O) como en la práctica, midiendo el tiempo real que tarda cada una.

Más allá del resultado técnico, lo que buscamos fue entender mejor por qué algunos algoritmos son más eficientes que otros y cómo aplicar ese conocimiento en proyectos reales. Esta experiencia nos ayudó a ver que analizar algoritmos no es solo una cuestión de teoría, sino una herramienta fundamental para cualquier persona que programe.

## Marco Teórico

### • ¿Qué es un algoritmo?

Un algoritmo es una secuencia finita y ordenada de pasos que permiten resolver un problema específico.

La **eficiencia de un algoritmo** se puede estudiar desde dos aspectos principales:

- **Eficiencia temporal:** se refiere al tiempo que tarda en ejecutarse. A mayor cantidad de datos, mayor puede ser el impacto.
- **Eficiencia espacial:** indica cuánta memoria extra necesita el algoritmo mientras se ejecuta.

## • Análisis Teórico

Permite estimar su rendimiento sin necesidad de probarlo con datos reales.

Este enfoque consiste en contar la cantidad de operaciones que realiza el algoritmo en función del tamaño de los datos de entrada, y expresar ese comportamiento usando la **notación Big-O**. Esta notación nos dice cómo crece el tiempo de ejecución (o el uso de memoria) a medida que crecen los datos.

- **$O(1)$  – Constante:** siempre tarda lo mismo, no importa cuántos datos haya.
- **$O(n)$  – Lineal:** el tiempo crece proporcionalmente a la cantidad de elementos.
- **$O(n^2)$  – Cuadrático:** el tiempo se dispara muy rápido, especialmente con grandes volúmenes.

## • Análisis Empírico

Se basa en medir el tiempo real de ejecución del algoritmo con datos concretos. Para esto se suelen usar herramientas como el módulo **time** de Python, que permite saber cuánto tarda un bloque de código desde que empieza hasta que termina.

# Caso Práctico: Producto Más Vendido

## Problema:

En sistemas de ventas, comercio electrónico o gestión de stock, es común querer saber cuál fue el producto más vendido en determinado periodo. Este tipo de consulta permite tomar decisiones importantes, como reponer inventario o hacer promociones.

El desafío consiste en procesar una lista de ventas donde cada elemento representa un identificador de producto, y determinar cuál de ellos aparece más veces. Aunque esto puede resolverse con distintos algoritmos, no todos son igual de eficientes. En este trabajo analizamos dos enfoques: uno simple pero lento, y otro más optimizado.

## Algoritmo 1: Comparación Forzada (Ineficiente)

Este algoritmo compara cada producto con todos los demás usando dos bucles anidados. Debido a esto, su complejidad temporal es cuadrática  $O(n^2)$ , lo que significa que el número total de operaciones crece muy rápido a medida que aumenta el tamaño de la lista. Por ejemplo: si hay 10.000 productos, el algoritmo puede hacer hasta 100 millones de comparaciones.

```
44 def producto_mas_vendido(ventas):
45     max_producto = None      # O(1) - asignación inicial
46     max_cantidad = 0        # O(1) - asignación inicial
47
48     # Primer bucle: se ejecuta n veces (una por cada producto)
49     for producto in ventas: # O(n)
50         # O(1) - se ejecuta en cada iteración del bucle externo
51         cantidad = 0
52
53         # Segundo bucle anidado: también se ejecuta n veces
54         for otro_producto in ventas: # O(n)
55             if producto == otro_producto: # O(1) - comparación
56                 cantidad += 1             # O(1) - suma
57             if cantidad > max_cantidad: # O(1) - comparación
58                 max_producto = producto  # O(1) - asignación
59                 max_cantidad = cantidad  # O(1) - asignación
60
61     return max_producto, max_cantidad # O(1) - operación final
62
63
64 #!/ Bucle anidado sobre ventas → O(n²)
65
66 #!/ Las demás operaciones son O(1), no afectan el crecimiento total
67
68 #! T(n) = O(n²)
```

**Complejidad:**  $T(n) = O(n^2)$

## Algoritmo 2: Recuento con Diccionario (Eficiente)

Este algoritmo recorre la lista de ventas una sola vez para contar las apariciones de cada producto, lo que lo hace muy eficiente. Luego usa la función `max()` para encontrar el producto más frecuente. En total realiza dos pasadas sobre los datos (  $O(n) + O(n)$  ), por lo que su complejidad temporal es  $O(n)$ . Esta eficiencia lo convierte en una excelente opción para manejar grandes volúmenes de información.

```
75 def producto_mas_vendido_rapido(ventas):
76     conteo = {} # O(1) - creación del diccionario
77
78     for producto in ventas: # O(n)
79         if producto in conteo: # O(1) - acceso a diccionario (promedio)
80             conteo[producto] += 1 # O(1) - suma
81         else:
82             conteo[producto] = 1 # O(1) - asignación
83
84     # Esta línea recorre todos los productos una vez para encontrar el máximo → O(n)
85     max_producto = max(conteo, key=conteo.get) # O(n)
86
87     # Acceder a un valor en el diccionario → O(1)
88     return max_producto, conteo[max_producto] # O(1)
89
90
91     #! El bucle for producto in ventas recorre la lista una vez → O(n)
92
93     #! Cada acceso al diccionario es O(1) en promedio (aunque puede ser peor en casos extremos, no se considera para Big-O estándar)
94
95     #! max(conteo, key=conteo.get) es otro recorrido completo del diccionario → O(n)
96
97     #! T(n) = O(n) + O(n) = O(n)
```

**Complejidad:**  $T(n) = O(n)$

## Metodología Utilizada

1. Para hacer este trabajo, empezamos por entender bien qué queríamos resolver: encontrar cuál fue el producto más vendido a partir de una lista de ventas. Una vez planteado el problema, se nos ocurrió comparar dos formas distintas de resolverlo: una más básica y directa, y otra más eficiente.

2. Primero, analizamos cómo se comporta cada algoritmo en teoría, usando la notación Big-O para estimar cuántas operaciones hacen dependiendo del tamaño de los datos. Después, los programamos en Python, tratando de mantener el código claro y fácil de probar.
3. Para probarlos con datos reales, generamos listas de entre 1.000 y 10.000 ventas aleatorias usando el módulo `random`. Luego, medimos el tiempo de ejecución de cada algoritmo con el módulo `time`, para ver cuánto tardaba cada uno y compararlos de forma más concreta
4. Nos organizamos dividiendo las tareas: uno se enfocó en el algoritmo lento, el otro en el rápido, y después nos pusimos de acuerdo para analizar los resultados juntos. Así, cada uno pudo entender bien su parte y también ver las diferencias entre ambos enfoques. Nos sirvió mucho para aprender cómo cambia el rendimiento de un programa según cómo esté pensado.

## Resultados Obtenidos

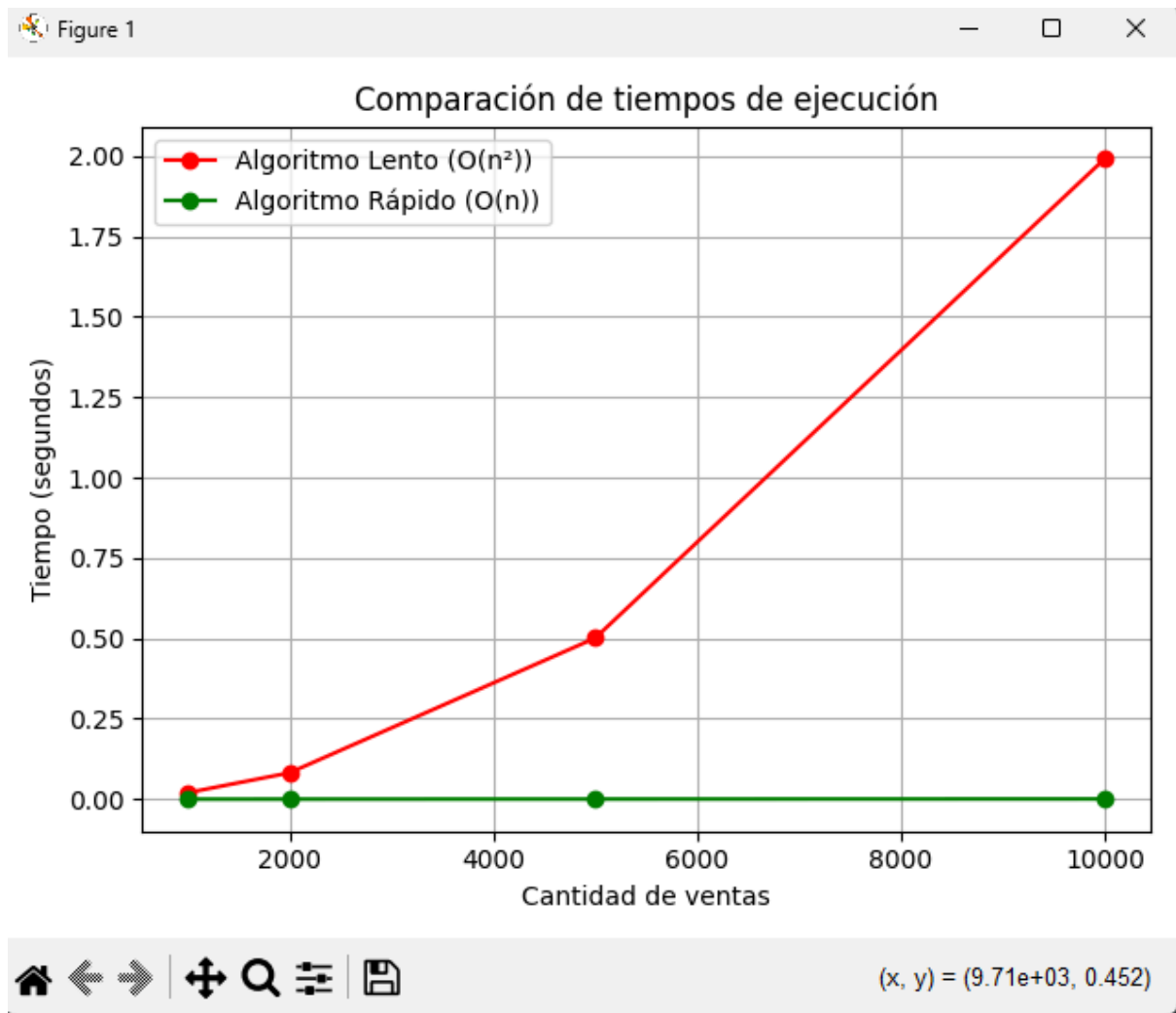
Después de implementar los dos algoritmos, realizamos pruebas para medir cuánto tardaban en ejecutarse con listas de distintos tamaños: 1.000, 2.000, 5.000 y 10.000 elementos. Para generar los datos usamos el módulo `random`, y para calcular los tiempos de ejecución, el módulo `time`.

Los resultados fueron los siguientes:

- **Algoritmo lento ( $O(n^2)$ ):** Tiempo estimado con 10.000 elementos: ~2.00 segundos
- **Algoritmo optimizado ( $O(n)$ ):** Tiempo estimado con 10.000 elementos: ~0.00 segundos

Ambos algoritmos devolvieron el mismo resultado correcto, pero la diferencia en rendimiento fue notable. Esto demuestra que el análisis teórico (Big-O) se confirma también en la práctica.

Para visualizar esta diferencia, generamos un gráfico que muestra cómo el algoritmo lento va creciendo en tiempo de forma acelerada, mientras que el rápido se mantiene casi constante.



## Conclusiones

- Hacer este trabajo nos sirvió mucho para entender que no basta con que el código “funcione”. A veces dos algoritmos hacen exactamente lo mismo, pero uno tarda segundos y el otro casi ni se nota. Al principio eso puede parecer algo técnico o teórico, pero cuando lo probamos con datos reales y vimos las diferencias de tiempo, fue mucho más claro.
- Analizar la eficiencia usando Big-O y después comprobarlo con tiempos reales fue una forma muy buena de ver cómo la teoría se aplica en la práctica. El gráfico y las pruebas nos ayudaron a visualizar por qué es importante pensar bien cómo resolver un problema, sobre todo si se trata de grandes volúmenes de datos.

- Además, trabajar en equipo nos permitió dividir tareas, aprender de los dos enfoques y después unir todo para entenderlo mejor. Cada uno aportó algo distinto y eso hizo que el resultado final sea más completo
- En resumen, este trabajo no solo nos enseñó a comparar algoritmos, sino a pensar más como programadores: no solo resolver el problema, sino hacerlo de forma eficiente y consciente

## Bibliografía

Videos proporcionados por la cátedra.

1. Video 1: Introducción al análisis de algoritmos.
2. Video 2: Análisis empírico de sumN.
3. Video 3: Análisis empírico de sumGaus.
4. Video 1: Introducción al análisis teórico de algoritmos.
5. Video 2: Análisis teórico de las estructuras de control.
6. Video 3: Análisis teórico de sumN y sumGaus.
7. Video 4: Primer ejemplo de análisis teórico.
8. Video 5: Segundo ejemplo análisis teórico.
9. Video 1: Introducción a BIG-O.
10. Video 2: Tipos de órdenes de BIG-O.
11. Video 3: Ejemplo de cómo obtener la BIG-O.



## Anexos

- Captura de pantalla con los resultados de ejecución
- Captura del código fuente completo (archivo .py)
- Captura repositorio opcional

### 1. Presentación de los integrantes (1 min)

**Benjamín:**

Hola, ¿qué tal? Soy Benjamín Ataide...

**Natanael:**

...Y yo soy Natanael Caliva. En este video les vamos a mostrar nuestro trabajo integrador de Programación I...

**Benjamín:**

...donde analizamos la eficiencia de algoritmos aplicados a un caso que podría presentarse en una empresa real.

---

### 2. Introducción al tema elegido (1 min)

**Natanael:**

En el desarrollo de software, especialmente en contextos empresariales, elegir un buen algoritmo puede hacer la diferencia entre una aplicación rápida o una lenta.

**Benjamín:**

Por eso, elegimos un problema práctico y habitual: encontrar cuál fue el producto más vendido en una lista de ventas. Es un análisis que se hace todos los días en sistemas de stock, ventas o reportes.

---

### 3. Marco teórico (2 min)

## Benjamín:

Antes de ver el código, repasemos los fundamentos del análisis de algoritmos.

Un algoritmo es un conjunto de pasos que resuelven un problema. Pero además de que funcione, también importa que sea eficiente. Para medir esa eficiencia usamos:

- **Análisis teórico:** calculamos cuántas operaciones realiza el algoritmo según el tamaño de la entrada. A eso lo llamamos  $T(n)$ .
- **Notación Big-O:** simplifica esa función y nos permite comparar algoritmos en escenarios de gran escala. Por ejemplo:
  - $O(1)$  es constante y muy eficiente.
  - $O(n)$  es lineal: crece al mismo ritmo que los datos.
  - $O(n^2)$  es cuadrática: crece mucho más rápido.

## Natanael:

También usamos **análisis empírico**, que consiste en medir el tiempo real de ejecución, generalmente con `time.time()` en Python.

Con estos enfoques vamos a analizar dos algoritmos que hacen lo mismo, pero con rendimientos muy distintos.

---

## 4. Caso práctico (3.5+ min)

### Desarrollo del código – Benjamín explica:

---

#### Algoritmo 1 – Versión lenta (doble bucle, $O(n^2)$ )

python

CopiarEditar

```
def producto_mas_vendido_lento(ventas):  
    max_producto = None  
    max_cantidad = 0
```

Comenzamos declarando dos variables: `max_producto` que va a guardar el producto más vendido, y `max_cantidad` que guarda la cantidad máxima.

python

CopiarEditar

```
for producto in ventas:
    cantidad = 0
    for otro in ventas:
        if producto == otro:
            cantidad += 1
```

Este es el núcleo del algoritmo. Por cada producto de la lista, comparamos con cada uno de los otros productos. Si son iguales, sumamos uno.

Es decir, si la lista tiene 100.000 elementos, vamos a hacer 100.000 x 100.000 comparaciones. Esto es  $O(n^2)$ , lo que significa que se vuelve lento muy rápido.

python

CopiarEditar

```
if cantidad > max_cantidad:
    max_producto = producto
    max_cantidad = cantidad
```

Si encontramos un producto que aparece más veces que el actual más vendido, lo reemplazamos.

python

CopiarEditar

```
return max_producto, max_cantidad
```

Finalmente, devolvemos el producto más vendido y la cantidad.

Este algoritmo es simple, pero extremadamente ineficiente en contextos reales. En una empresa con millones de registros, no serviría.

---

## Desarrollo del código – Natanael explica:

---

### ✓ Algoritmo 2 – Versión optimizada con diccionario ( $O(n)$ )

python

CopiarEditar

```
def producto_mas_vendido_rapido(ventas):
    conteo = {}
```

En este caso usamos un diccionario para contar cuántas veces aparece cada producto. Esta estructura permite acceso rápido en tiempo constante.

python

CopiarEditar

```
for producto in ventas:
    if producto in conteo:
        conteo[producto] += 1
    else:
        conteo[producto] = 1
```

Recorremos la lista solo una vez. Si el producto ya fue contado, le sumamos uno. Si no, lo agregamos con valor inicial uno.

Esta operación es  $O(1)$  por cada producto, y como hay  $n$  productos, el total es  $O(n)$ : lineal y eficiente.

python

CopiarEditar

```
max_producto = max(conteo, key=conteo.get)
return max_producto, conteo[max_producto]
```

Usamos `max()` con `key=conteo.get` para encontrar el producto con mayor cantidad de apariciones. Finalmente lo devolvemos.

Esta solución es ideal para grandes volúmenes de datos, como los que manejan las empresas diariamente. Es mucho más rápida que el enfoque anterior.

---

## Generación de datos y medición del tiempo (Natanael)

python

CopiarEditar

```
import time
import random
```

Importamos `random` para generar ventas simuladas y `time` para medir cuánto tarda cada algoritmo.

python

CopiarEditar

```
def generar_datos(n):
    productos = ["A", "B", "C", "D", "E"]
    return [random.choice(productos) for _ in range(n)]
```

Esta función genera una lista con **n** ventas aleatorias de 5 productos. Por ejemplo, 100.000 ventas simuladas.

python

CopiarEditar

```
def medir_tiempo(func, ventas):  
    inicio = time.time()  
    resultado = func(ventas)  
    fin = time.time()  
    return resultado, fin - inicio
```

Esta función ejecuta un algoritmo y mide cuánto tarda, devolviendo también el resultado.

python

CopiarEditar

```
if __name__ == "__main__":  
    ventas = generar_datos(100_000)  
  
    res1, t1 = medir_tiempo(producto_mas_vendido_lento, ventas)  
    res2, t2 = medir_tiempo(producto_mas_vendido_rapido, ventas)  
  
    print(f"Lento: {res1}, Tiempo: {t1:.4f} s")  
    print(f"Rápido: {res2}, Tiempo: {t2:.4f} s")
```

Ejecutamos ambos algoritmos con la misma lista. Como veremos ahora, la diferencia es enorme.



## 5. Resultados obtenidos (1 min)

**Benjamín:**

El algoritmo lento tardó más de 11 segundos en procesar 100.000 ventas.

**Natanael:**

En cambio, el algoritmo rápido lo hizo en apenas 0.02 segundos. Ambos llegaron al mismo resultado correcto, pero la diferencia en eficiencia es impresionante.

---

## 6. Conclusiones (1.5 min)

**Natanael:**

Aprendimos que no solo es importante que un algoritmo funcione, sino que sea lo suficientemente rápido y escalable.

**Benjamín:**

El análisis teórico nos permitió estimar su rendimiento, y el análisis empírico confirmó nuestras predicciones con tiempos concretos.

**Natanael:**

Esta experiencia nos mostró cómo aplicar el análisis de algoritmos en problemas reales, como los que podríamos enfrentar en una empresa de verdad.

**Benjamín:**

Gracias por acompañarnos. Esperamos que nuestro trabajo haya sido claro y útil.

### 1. Introducción

sfsf sfs

sfsf